

PRD: QA Signal Loop

Status: Design case study. Prototype only, not deployed.

1. What it is

QA Signal Loop reads the bug reports a team already writes, groups the ones that are actually the same problem, and shows the Product Manager which problems keep coming back.

It does not create bugs, assign work, manage tickets, or write tests. It reads what exists and produces one thing: a short ranked list for the PM each cycle.

2. The problem

Bug reports get written so someone can fix the bug. After the fix, nobody reads them again.

So a team can keep shipping the same kind of problem for months without noticing. The information is there - it's just spread across hundreds of tickets nobody looks at together.

3. Where it came from

I ran QA on a suite of 2,000+ test cases over several monthly cycles. The workflow was:

Test cases lived in a spreadsheet, one sheet per module. Failures were logged in Notion - a different tool - with the tester, date, device, a description and a screenshot. The PM assigned them to engineers. Engineers fixed and closed. **There was no formal re-testing step**, and nobody opened the log again afterwards.

Across cycles, the same kind of problem kept coming back - tied to a component several flows shared, so it showed up as separate tickets from different sheets in different testers' words.

Every one was handled correctly on its own. The repeat was invisible, because nobody's job was to look at the log as a whole.

The four gaps:

1. No re-testing - "fixed" and "confirmed fixed" meant the same thing
2. No way to see it together - the log was a to-do list, never data
3. No memory between tickets - nothing connected a new bug to an identical old one
4. Nothing reached Product - the data helped one fix and stopped there

4. Who it's for

The PM. Not the QA lead, not the engineer.

QA and engineers already see every bug daily. The person missing the bigger picture is the one deciding what the team works on next.

That choice decided everything else:

Because it's for a PM	So it
They decide what gets worked on	Sorts by how much users are affected, not bug count
They'll open it once a cycle	Sends a short list rather than waiting to be searched
They have to explain it to an eng lead	Uses rules simple enough to explain in one sentence
They prioritise, they don't diagnose	Stops at showing the problem - no fixes, no estimates

5. How it works

The data

The old log had who, when, what device, a description, a screenshot. Enough to fix one bug. Not enough to spot a pattern across four hundred.

Three things were missing: a link from the bug to the test case that found it, a separation between *where you saw it* and *what actually broke*, and any record of *why* it happened.

The rule: simple where a human types, structured where a machine reads.

- **Tester** answers what they can actually see - what were you trying to do, what happened, could you continue. Two new dropdowns, ~5 seconds.
- **LLM** guesses the component and bug type from that description.
- **Engineer** confirms those and adds why it happened and whether the fix touched shared code. Two pre-filled dropdowns, ~5 seconds.
- **PM** tags each module once as main / supporting / minor path. ~15 minutes total.

Asking testers to name the component would have produced guesses, and a guess you can't identify later is worse than a blank. Full field list is in the data model reference.

The honest cost: the engineer spends time on a field that helps the PM, not them. That's why it's a pre-filled dropdown and not a text box.

The grouping

The lazy version groups by module and shows you that the biggest module has the most bugs. Useless - the PM already knew that.

It has to separate two things that look identical in a plain list: a big module with 40 bugs because it's big, and a weak spot with 12 bugs that are all the same failure under different ticket numbers.

Step 1 - rules. Same component, same cause, same type, within three cycles. Browser isn't part of the match, because a layout bug hitting Safari and Chrome is one problem, not two.

Step 2 - the LLM. Step 1 only works when the fields are right, and "which component" often isn't. So the LLM reads the free-text descriptions of everything step 1 missed and asks one question: do any of these describe the same failure? A suggested merge has to come with a reason, and it's labelled in the interface so the PM can check it.

Rules run first, and that's about trust. A PM told "these seven bugs are the same problem" will act on it once. If it's wrong, never again.

Making the list: three or more bugs, spread across at least two cycles. Or two, if a fix touched shared code. Or one, if it was reopened.

Ordering: repeats × path importance (3/2/1) × how blocking (2/1.5/1). A signup problem with 3 blocking bugs scores 18; a settings problem with 7 cosmetic ones scores 7. Blunt whole numbers on purpose, so the PM can explain the order.

Where AI is used

One job: reading messy free text and spotting that different descriptions mean the same thing. "Login button doesn't respond on mobile", "auth CTA unresponsive, Safari iOS", "can't sign in from phone" - same failure, tagged three different ways, no shared keywords. That's what LLMs are actually reliable at.

Plus pre-filling the engineer's two dropdowns, to make them nearly free.

Not used for: writing sprint summaries (the log already went unread - producing a paragraph nobody reads about data nobody reads is the same problem with an API bill attached), setting priority (that depends on roadmap and customers, none of which is in a bug report), predicting future bugs (needs training data that won't exist for a year), or a chat interface (turns something that comes to you into something you have to remember to ask).

The line: the LLM tidies up language, it doesn't make judgements. Every real decision is a rule you can read.

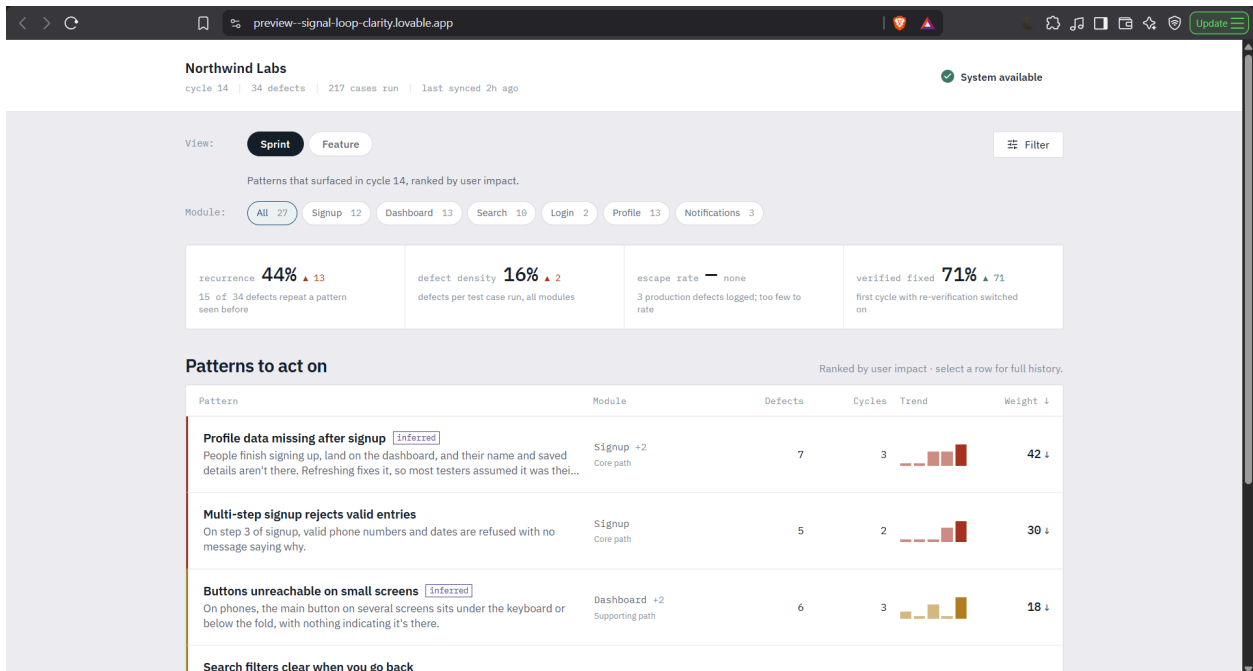
The four numbers

1. **Bugs per test case, by module.** Dividing is the point - raw counts just rank modules by size. Only possible because of the link to the test suite.
2. **Repeat rate.** What share of bugs repeat something seen before. 40 bugs at 10% means new ground; the same 40 at 45% means the problem is fix quality, not bug volume. Those look identical in a plain list and need opposite responses.
3. **Bugs that reached production. Probably not calculable** with most teams' data, since production bugs arrive through support, not the log. So it shows as a blank with the reason, not a confident percentage built on three data points.
4. **How long a pattern went unnoticed.** This measures the system, not the product. If it doesn't come down, the system isn't working.

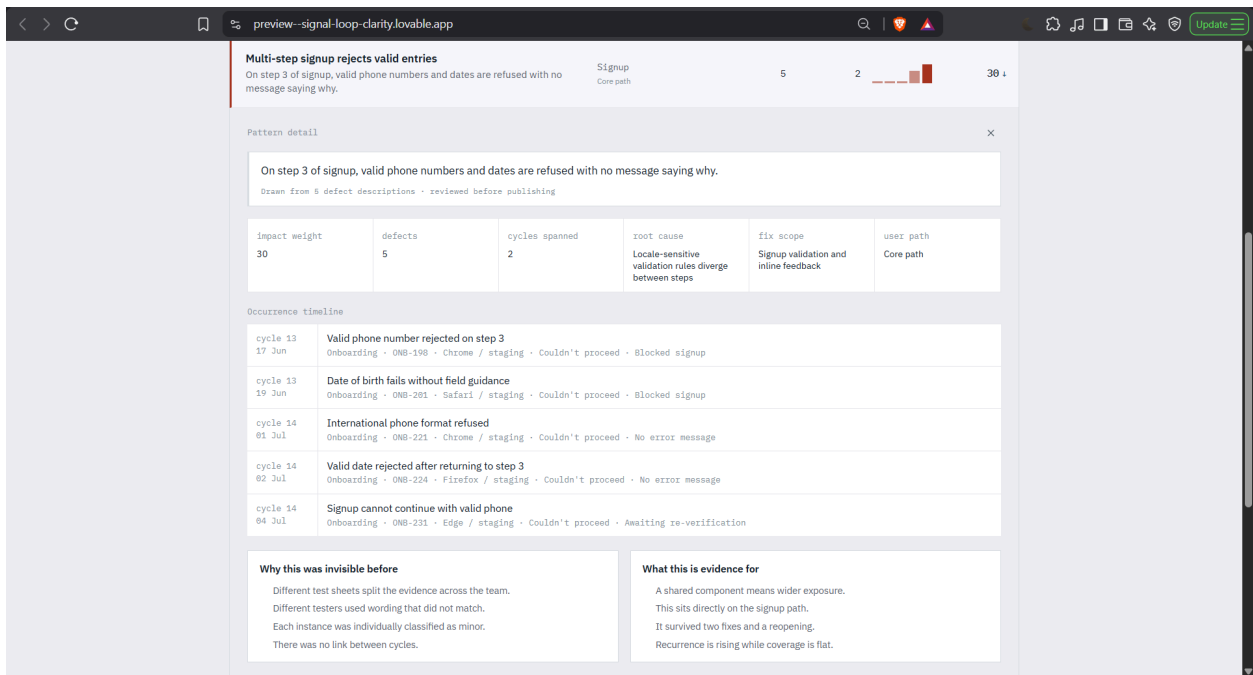
Left out: time-to-fix (an engineering speed metric, wrong tool) and per-tester bug counts (turns a diagnostic tool into surveillance, and the whole thing depends on honest input).

6. What it looks like

One page. A short strip of numbers at the top, then a ranked list of problems - one row each, with a plain sentence about what a user actually experiences and five small bars showing which cycles it appeared in. Clicking a row expands it underneath rather than opening a new page.



Sprint / Feature toggle. Sprint shows what turned up this cycle. Feature shows everything by product area regardless of whether it appeared this cycle, so a problem that went quiet for a sprint doesn't disappear. That second view is what catches something coming back after a few months.



7. What I decided not to build

- **No ticket management.** The most important one. The moment it writes back, it competes with the tool the team already uses and adopting it becomes a migration instead of an addition.
- **No test writing.** Changes the user from PM to QA lead, and something built for both serves neither.
- **No CI setup.** Needs engineering work *before* the system produces anything, and something that needs a sprint of setup won't get that sprint.
- **No fix suggestions.** Would need codebase access and answers a question the PM never asked.
- **No importing old bug data.** It would make a much better demo. But old records don't have why-it-happened or what-the-fix-touched, so the rules can't run and the LLM would be guessing. If the first thing a PM sees looks confident and turns out wrong, they won't trust it again.

The thread: each one protects a single path - existing bug data in, one useful signal out.

8. Rollout and success

Phase 1 - add the fields and the re-testing step to the existing log. No dashboard. Two cycles to build up proper data. The re-testing step is worth doing on its own. **Phase 2** - run the grouping and check the results by hand with the PM and QA lead. Get the thresholds right before anyone sees a dashboard. **Phase 3** - ship the dashboard, track how many groups get dismissed.

A dashboard that ships with the wrong thresholds loses credibility in its first cycle and doesn't get a second.

Success: fields filled on 80%+ of closes by cycle 2. Patterns spotted within 2 cycles by cycle 4. And by cycle 6, at least one decision per cycle that traces back to the list - that's the real test.

The failure case: if the PM reads the list, agrees with it, and changes nothing, it hasn't helped, even if the grouping is perfect. It would just be the same ignored data, shown a bit better.